Design Patterns

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**



Structural

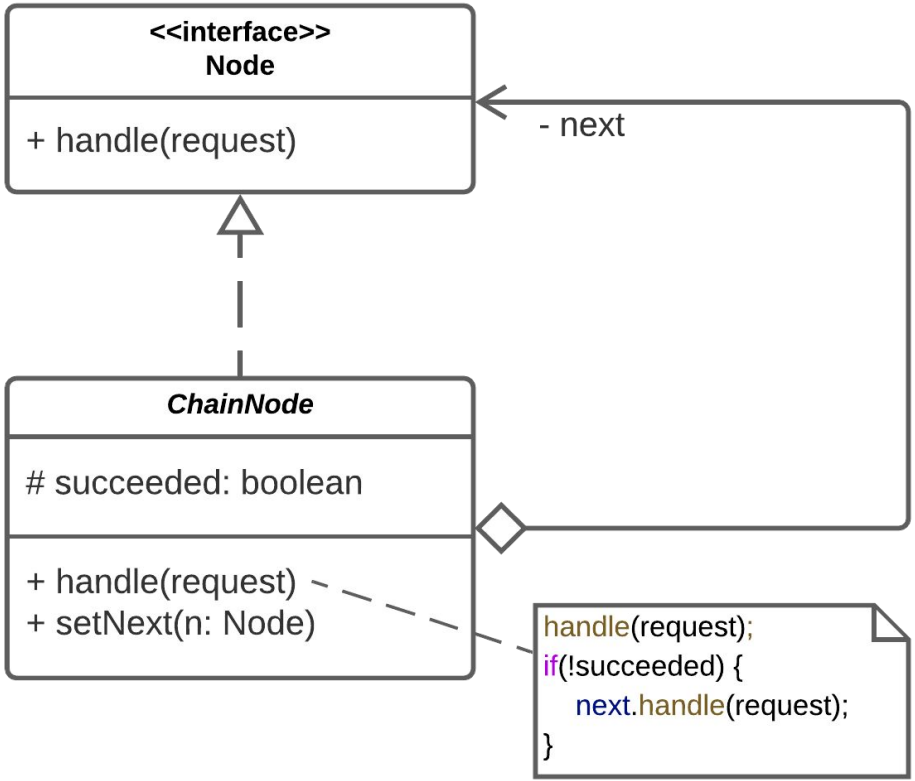
- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight



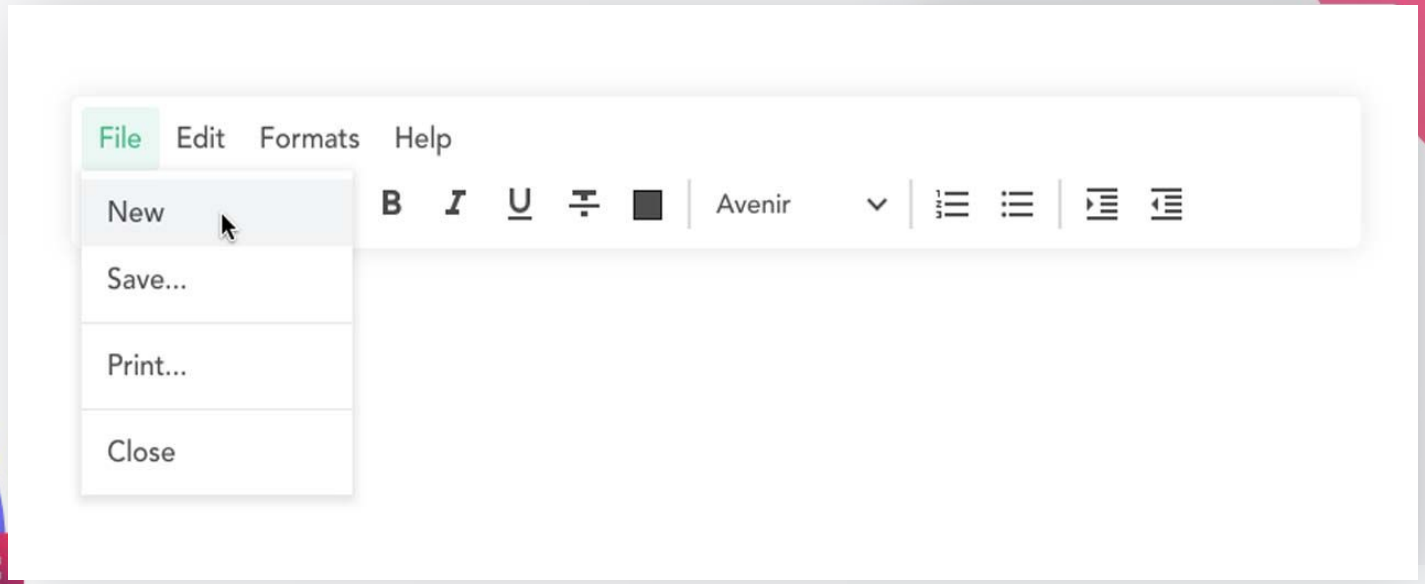
Behavioral

- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State

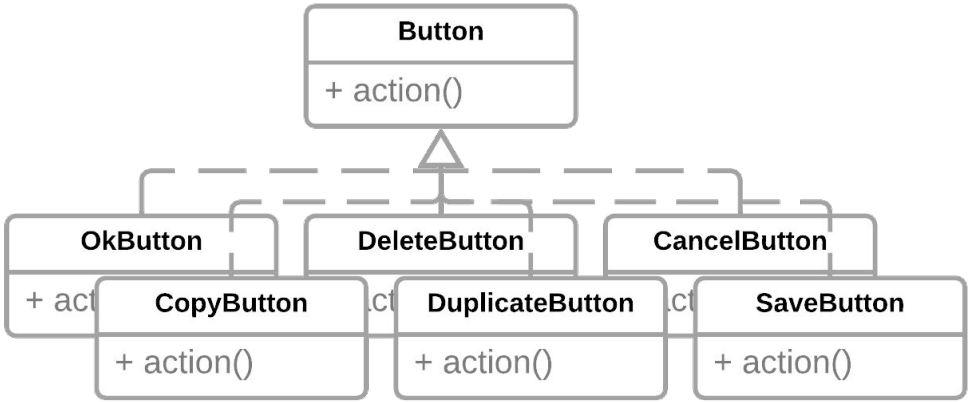
Chain Of Responsibility



The UI Problem



Extend the button?...

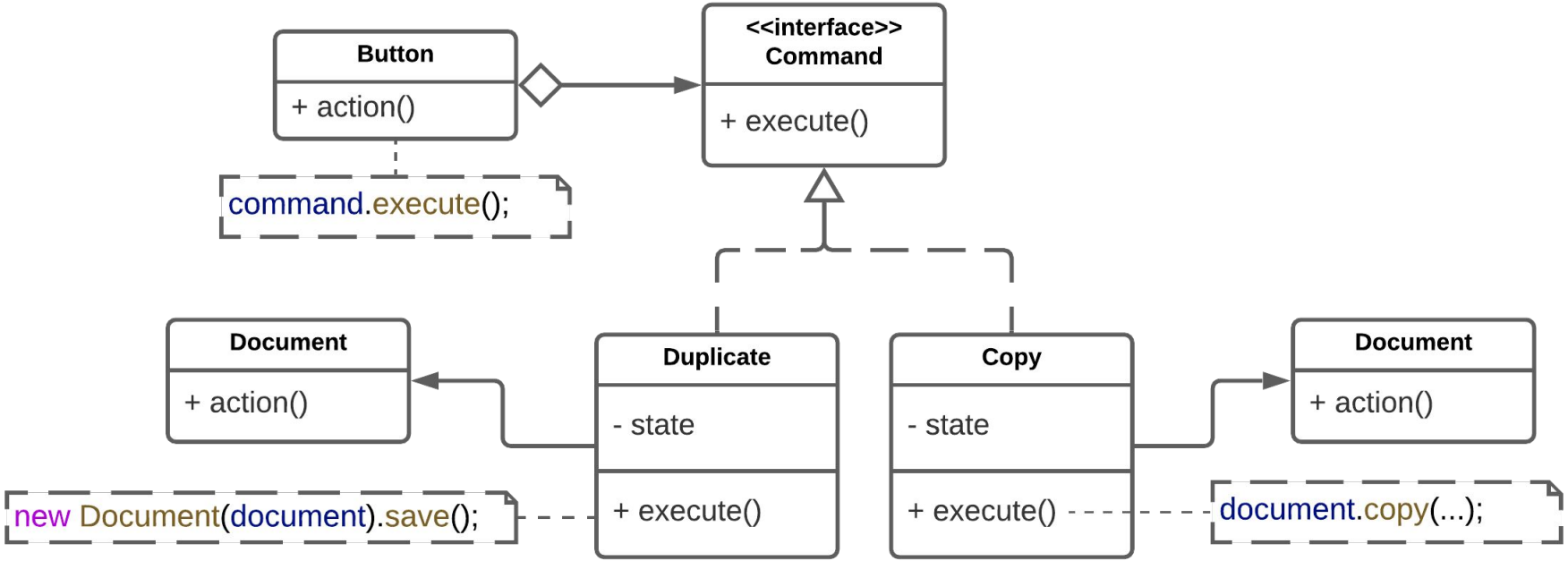




Favor composition over inheritance!

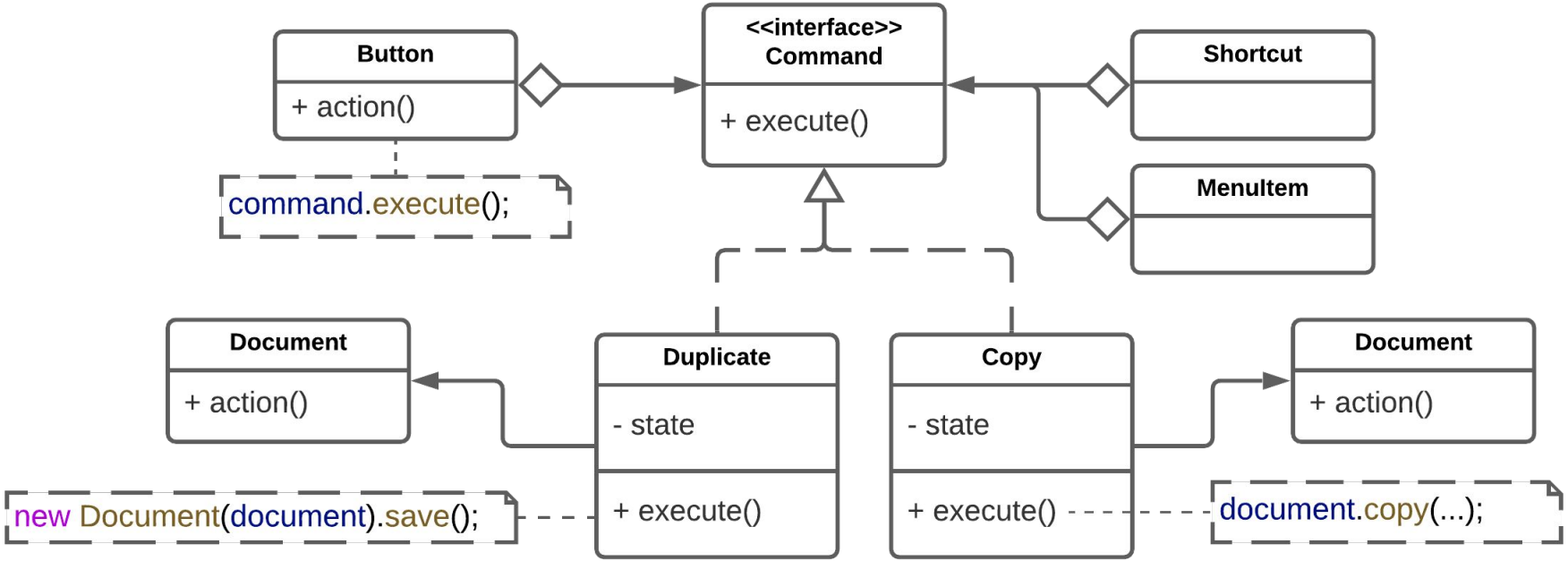
Add as many Commands as we need...





Command can be reused!





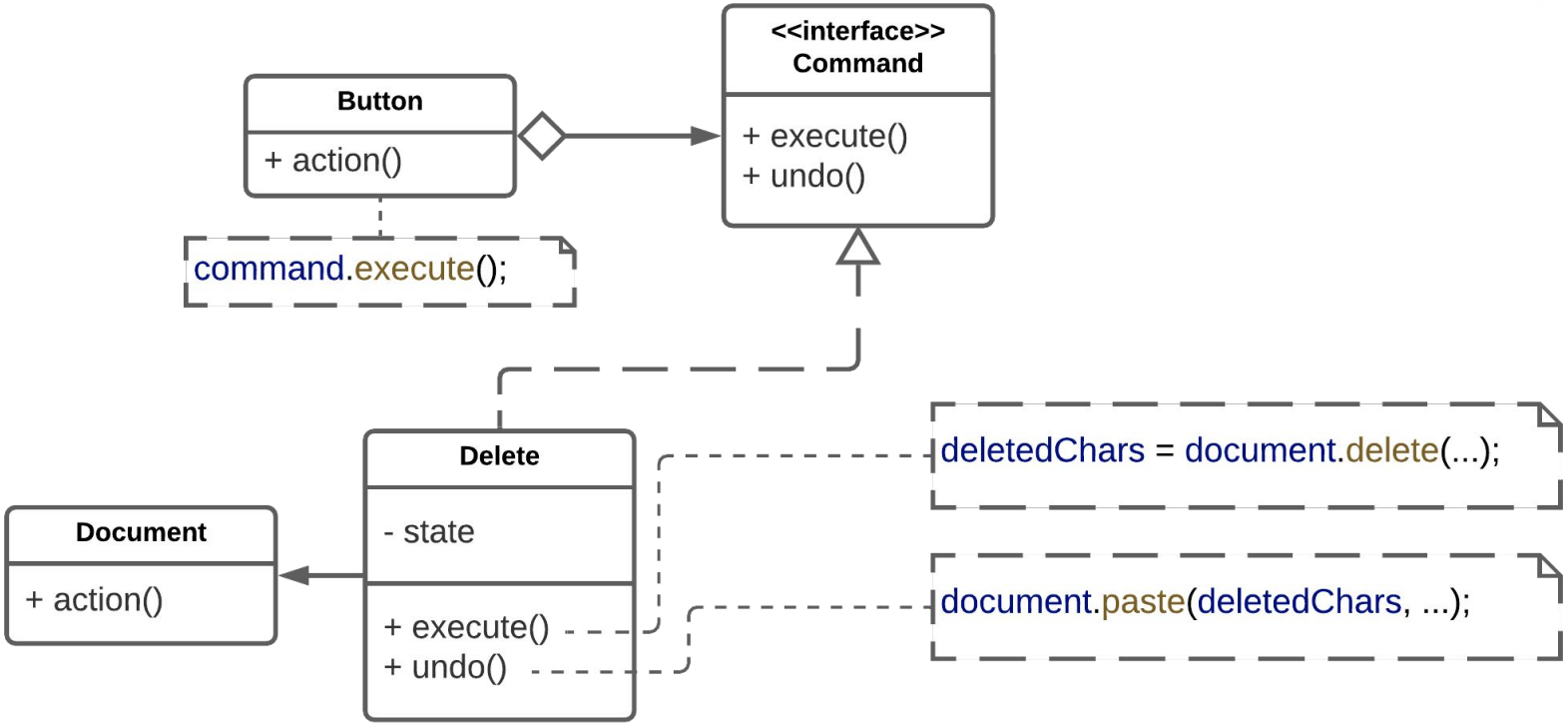
Commands can be queued!



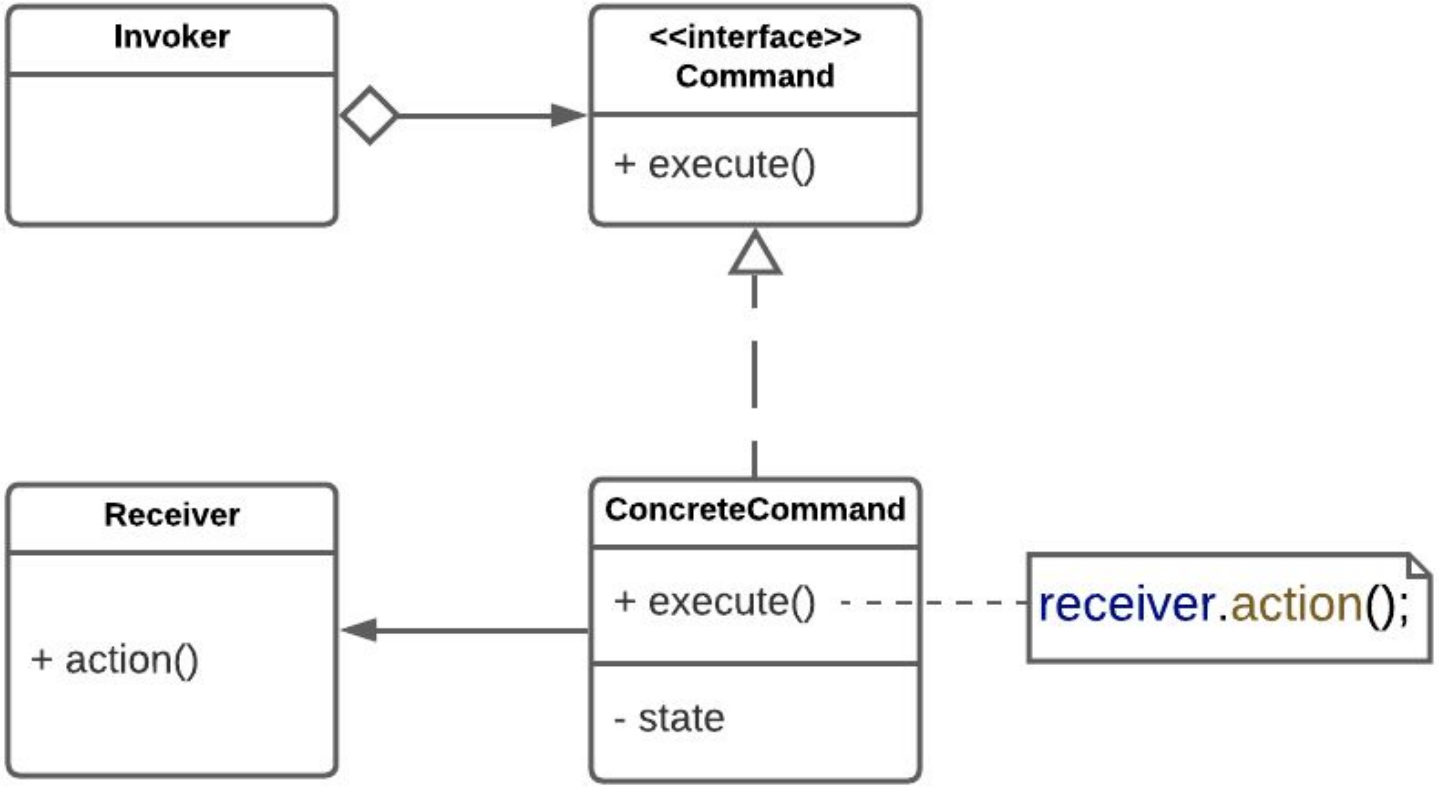


Commands can be undone...



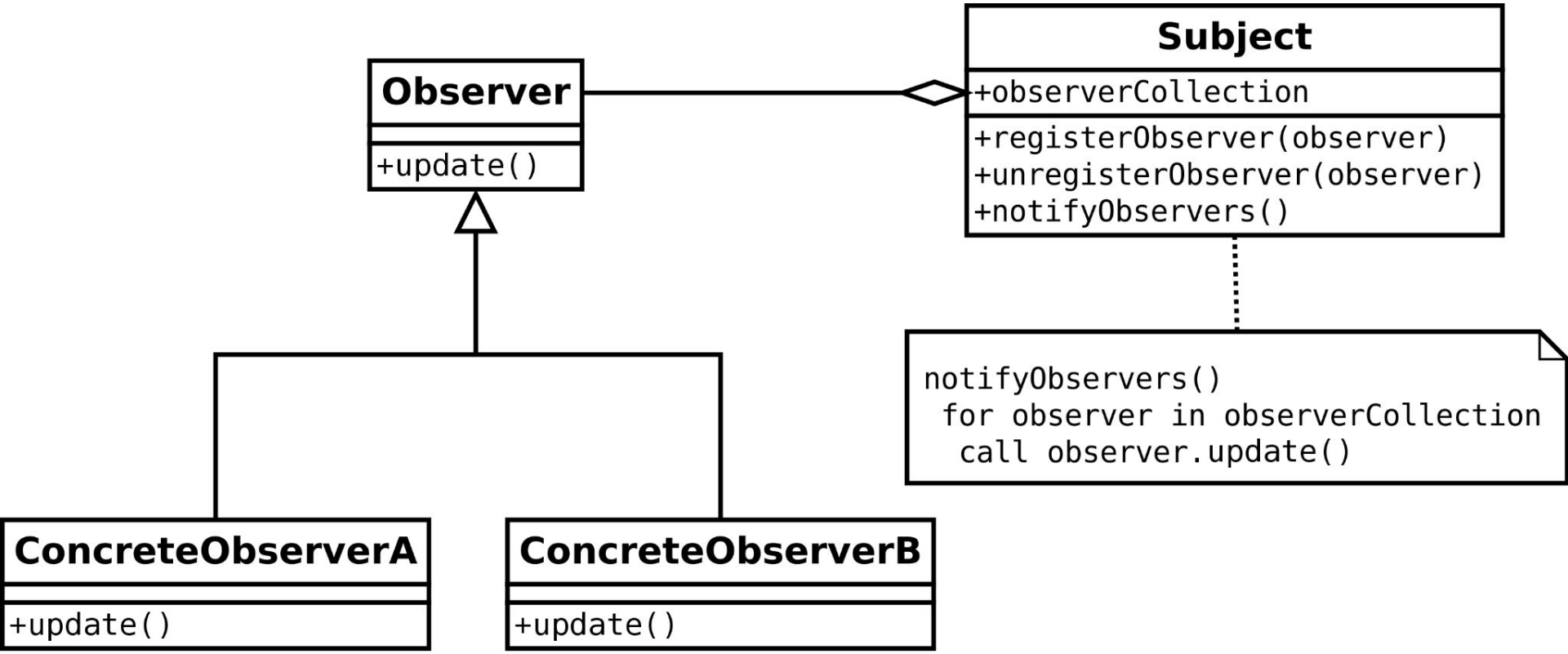


Command Pattern



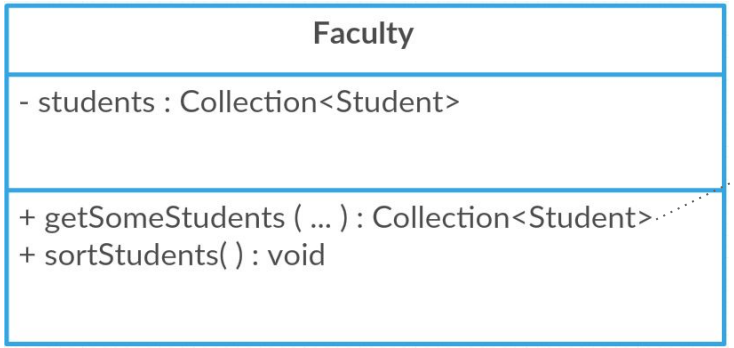
The Observer Pattern




Observer
+update()
Subject
+observerCollection
+registerObserver(observer)
+unregisterObserver(observer)
+notifyObservers()
notifyObservers()
for observer in observerCollection
call observer.update()
ConcreteObserverA
+update()
ConcreteObserverB
+update()

Inheritance vs composition

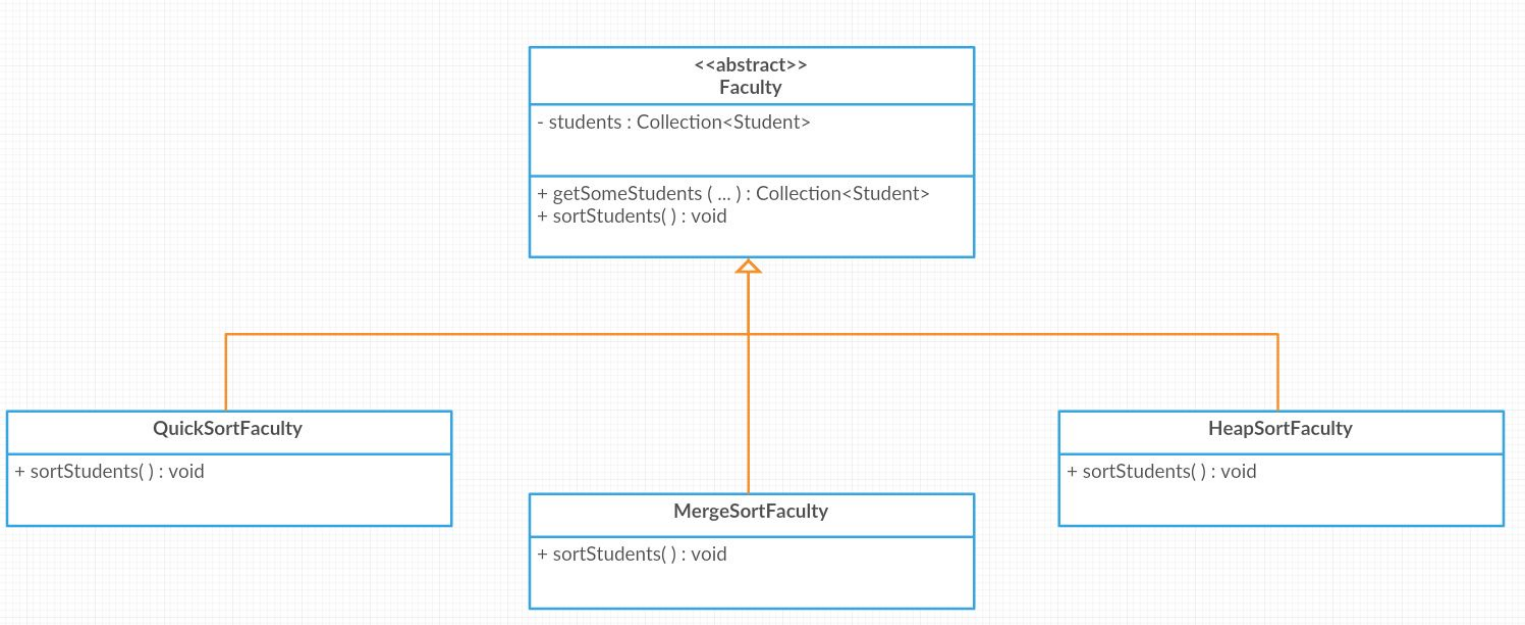
Example




```
public Collection<Student> getSomeStudents()
{
  //do some work
  sortStudents();
  //do some work
  return ...;
}
```

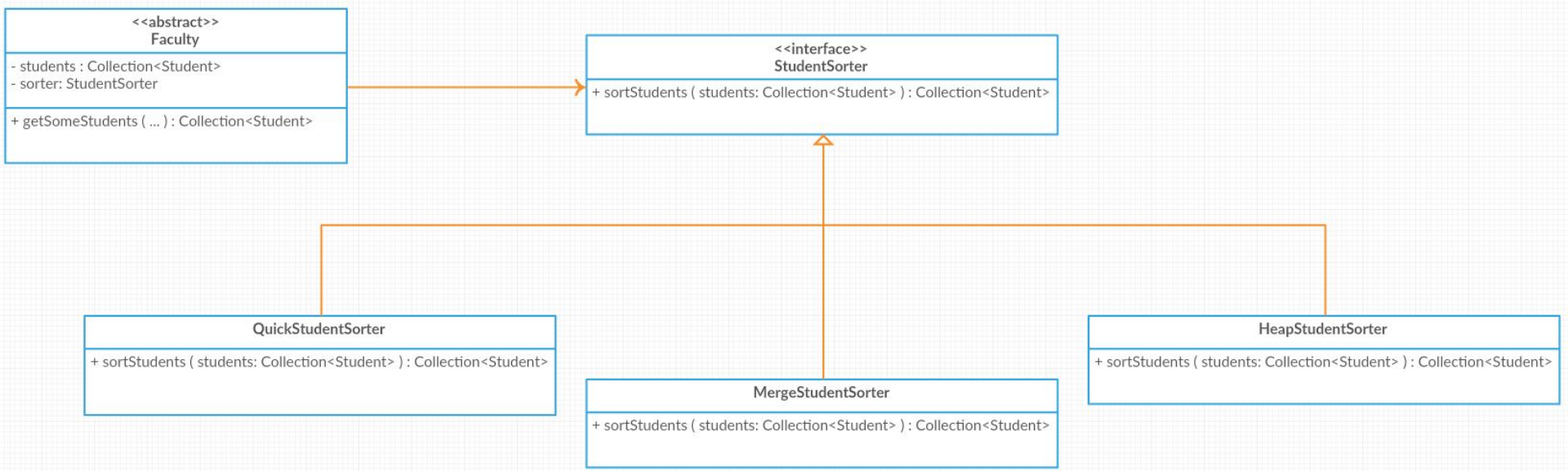
What if you want to be able to sort in multiple ways? QuickSort, MergeSort,...

Solution?



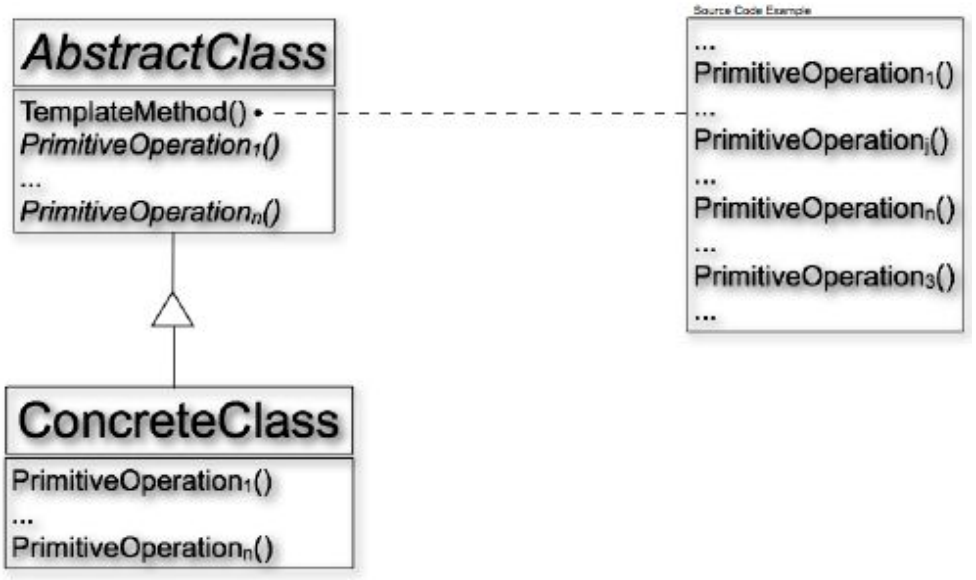
Ok...but what if some time I want Quicksort, some other time Mergesort and another time, another sort...?

Solution

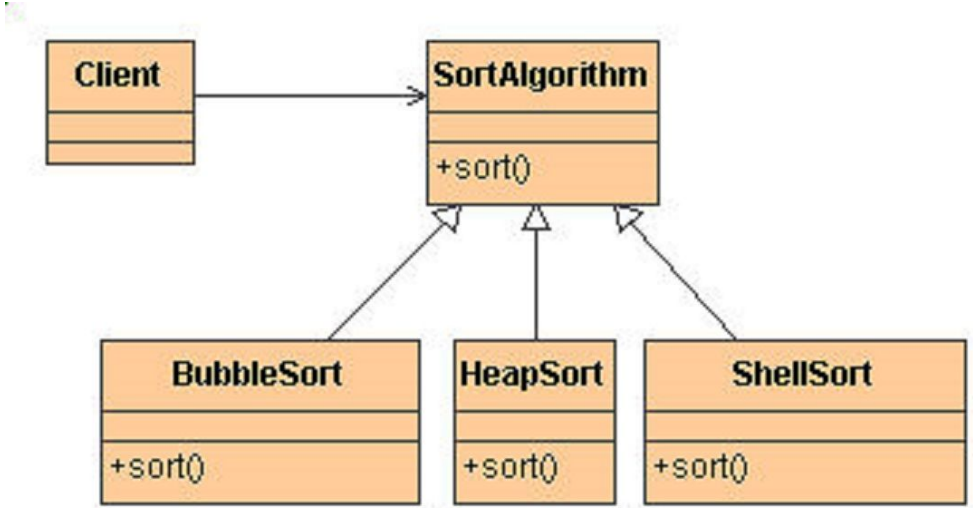

<<abstract>>
Faculty
- students : Collection<Student>
- sorter: StudentSorter
+ getSomeStudents (...) : Collection<Student>
<<interface>>
StudentSorter
+ sortStudents (students: Collection<Student>) : Collection<Student>
QuickStudentSorter
+ sortStudents (students: Collection<Student>) : Collection<Student>
MergeStudentSorter
+ sortStudents (students: Collection<Student>) : Collection<Student>
HeapStudentSorter
+ sortStudents (students: Collection<Student>) : Collection<Student>

Template method
vs
Strategy

Template method pattern



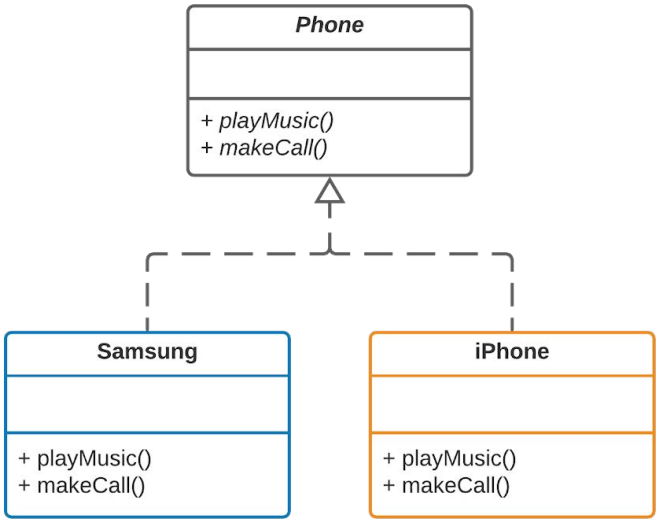
Strategy pattern



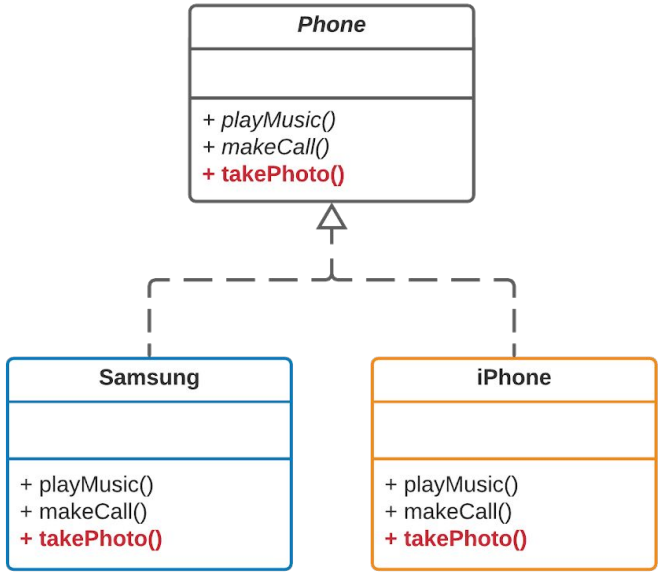
The Phone Apps Compatibility Problem



The Phone Apps Compatibility Implementation



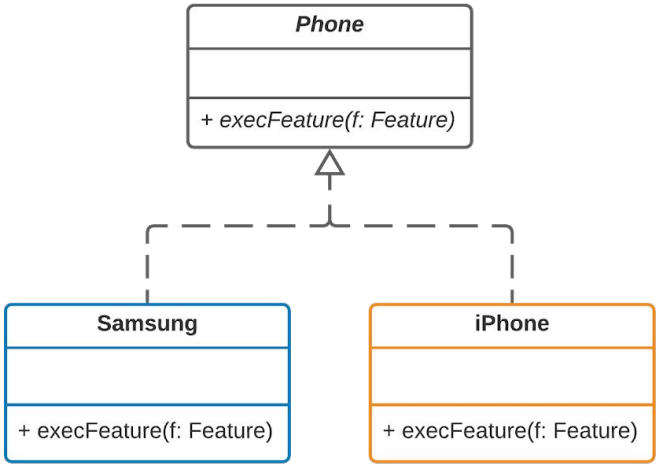
How do you add new features?





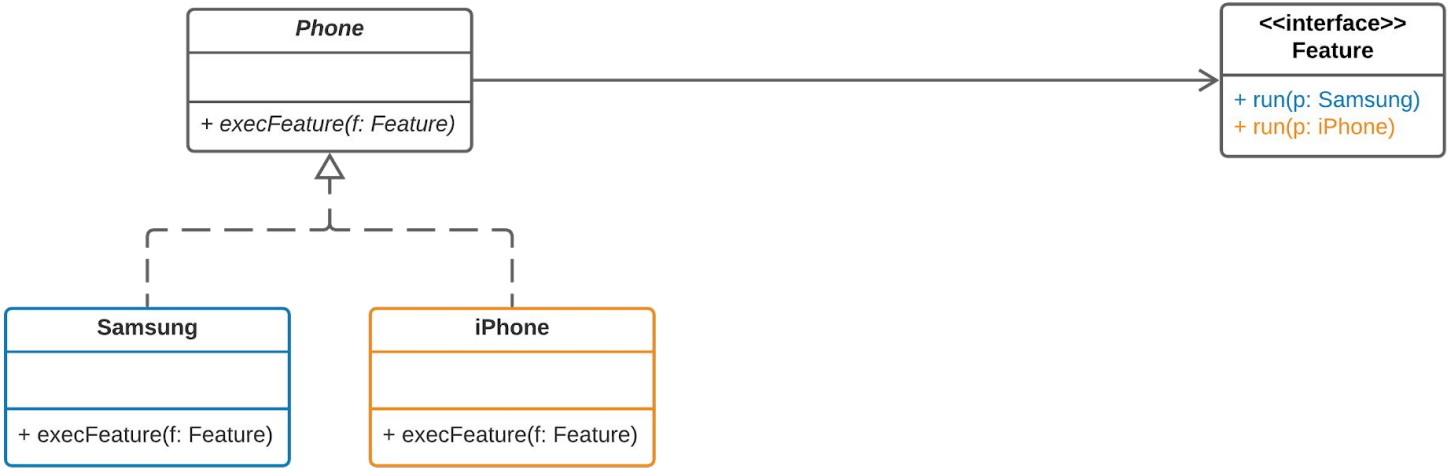
Use only one method?...





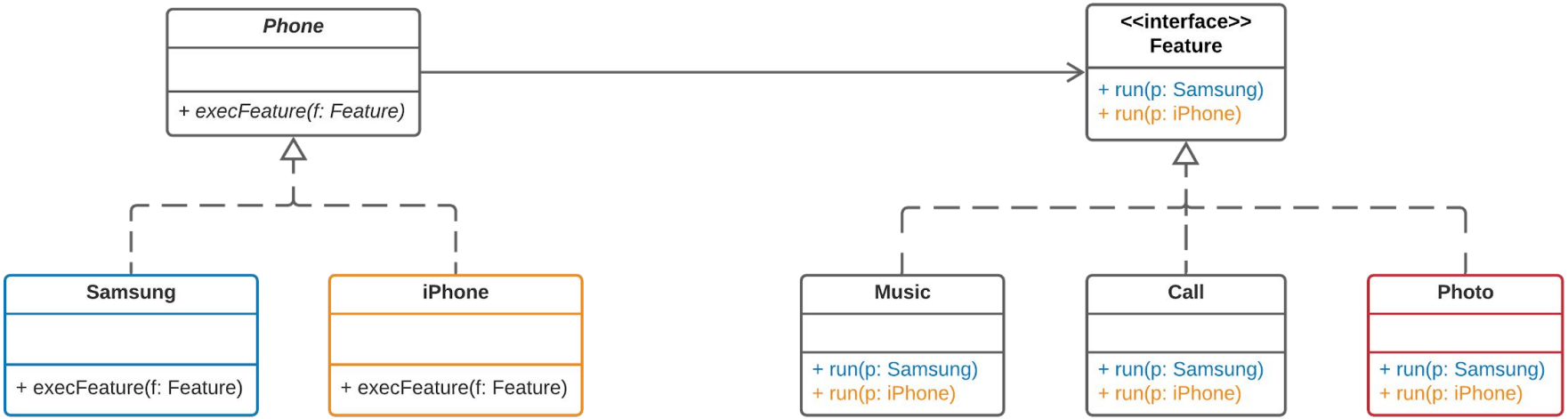
2 run Methods, one for each phone...





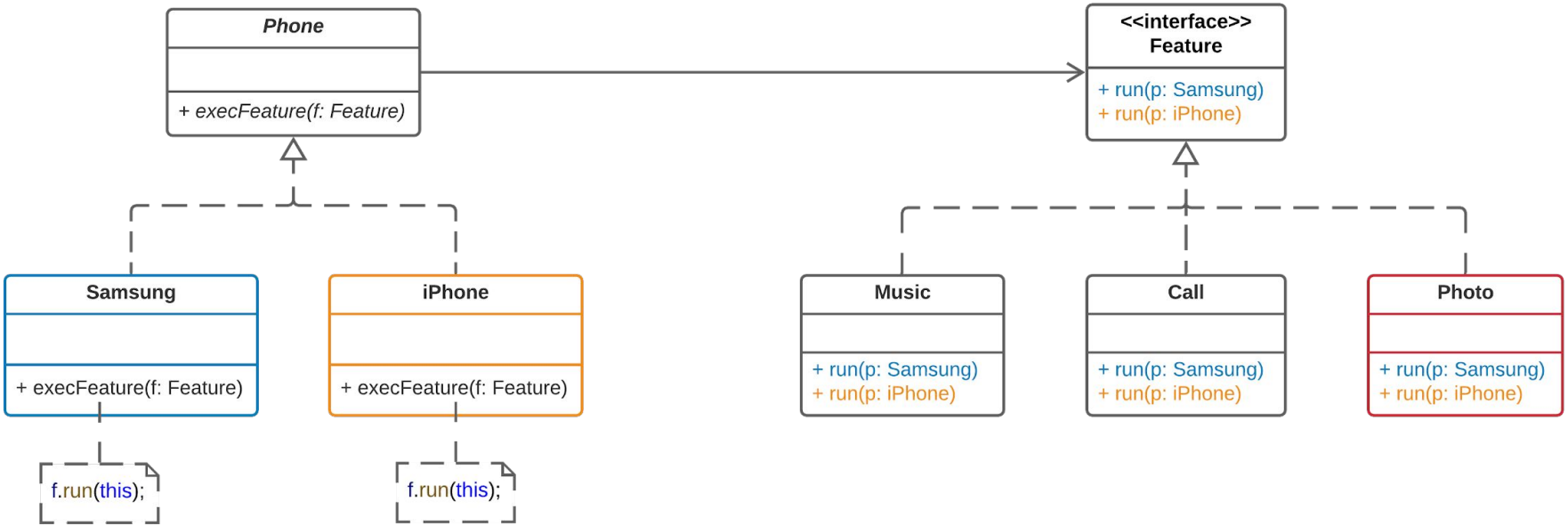
Now we can easily add new features...





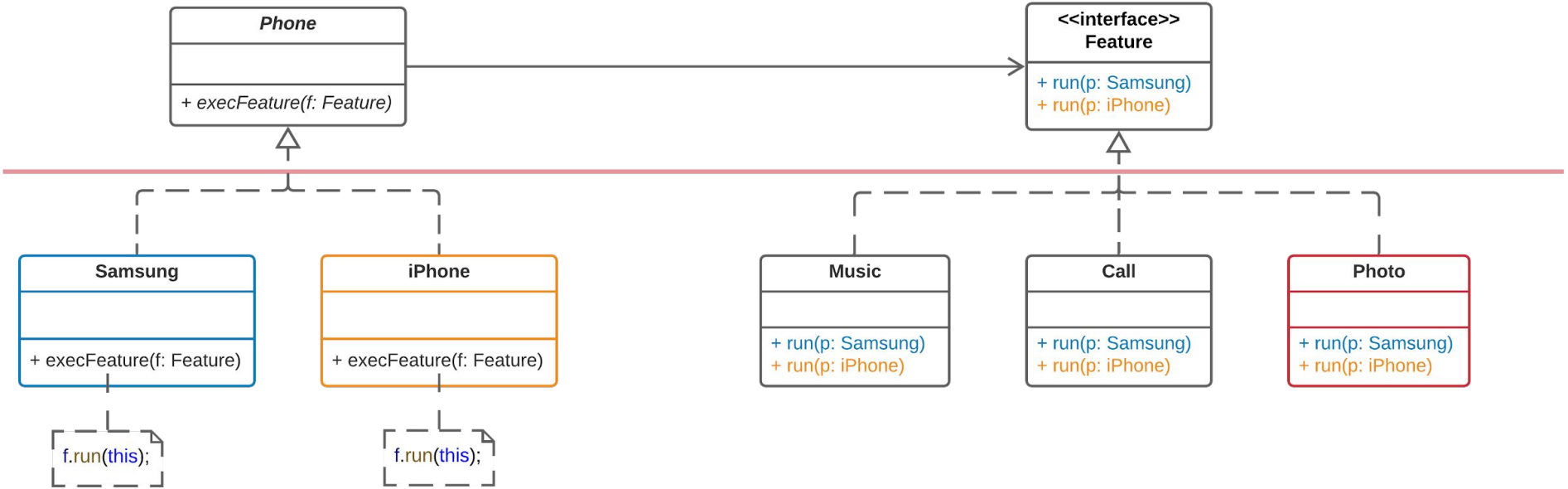
And they connect easily...





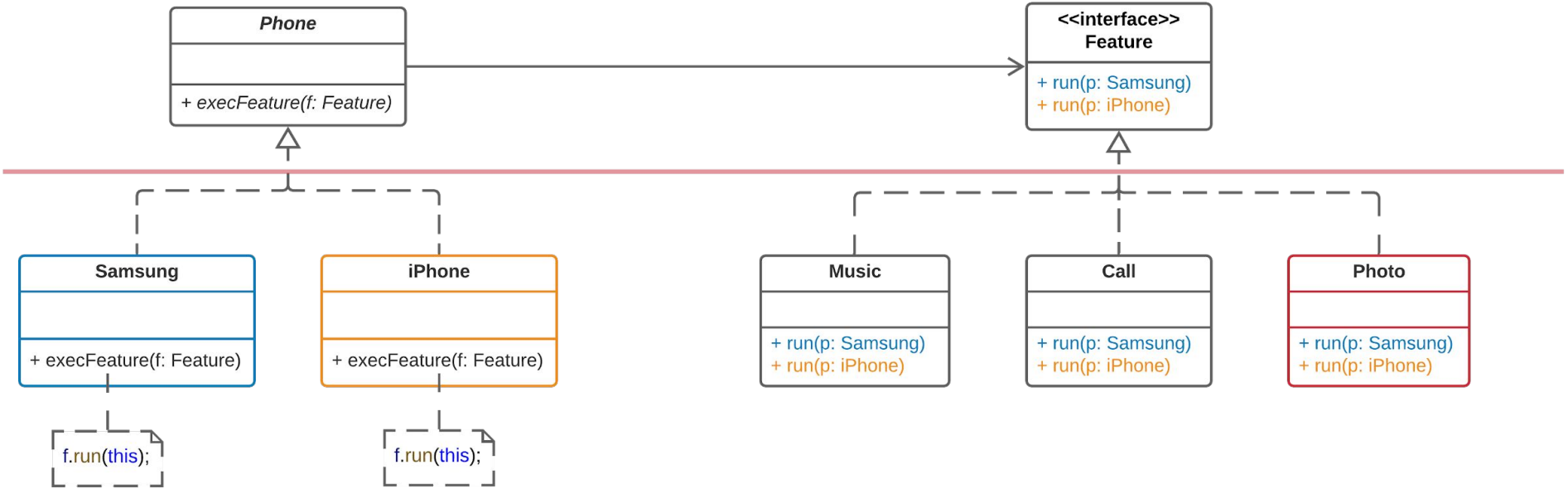
The Visitor Pattern





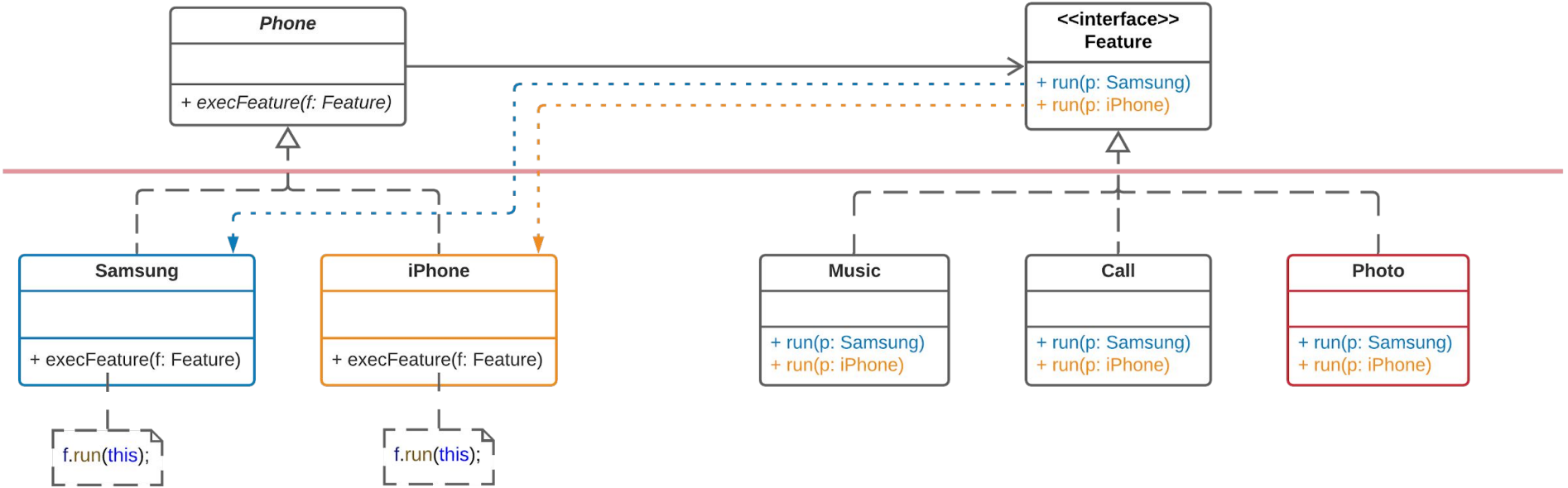
But is it perfect?





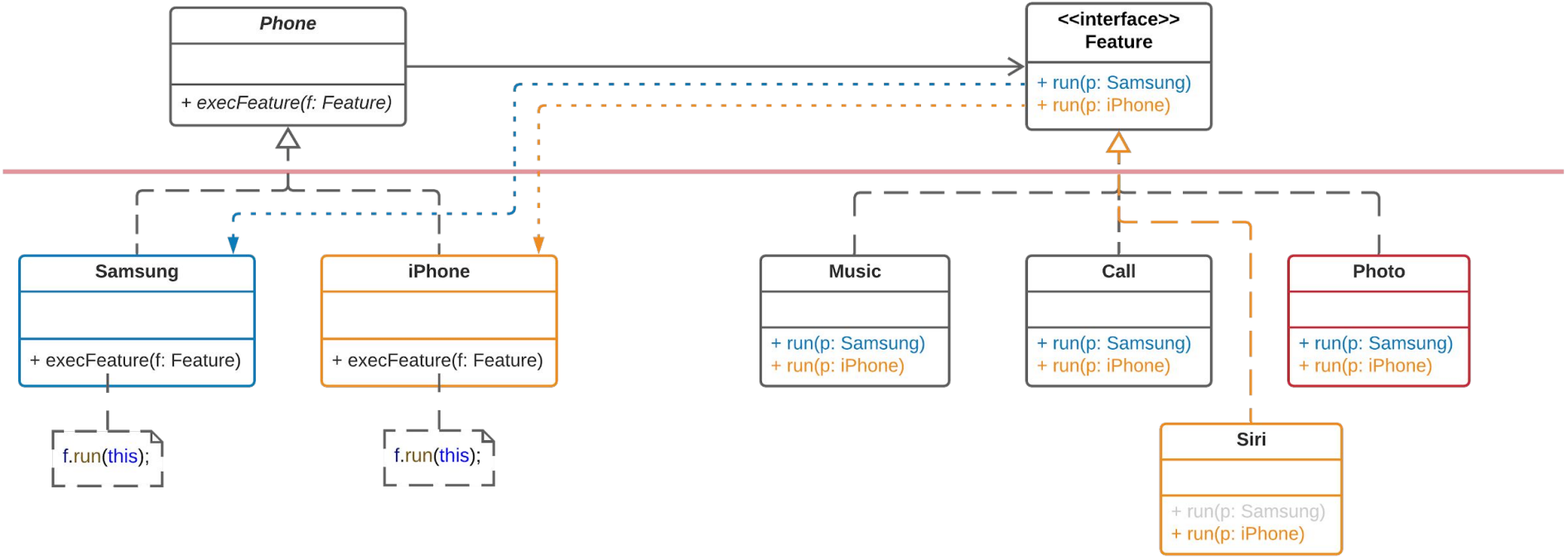
There is a subtle cyclic dependency...





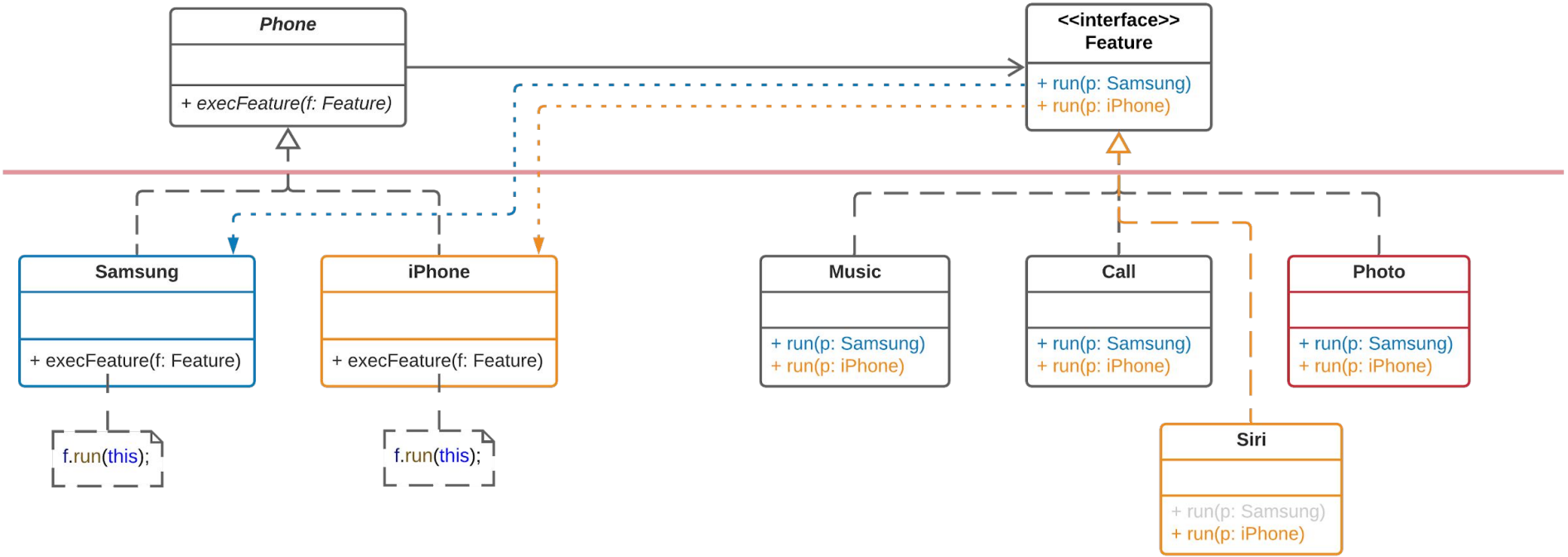
All features need to support both phones!





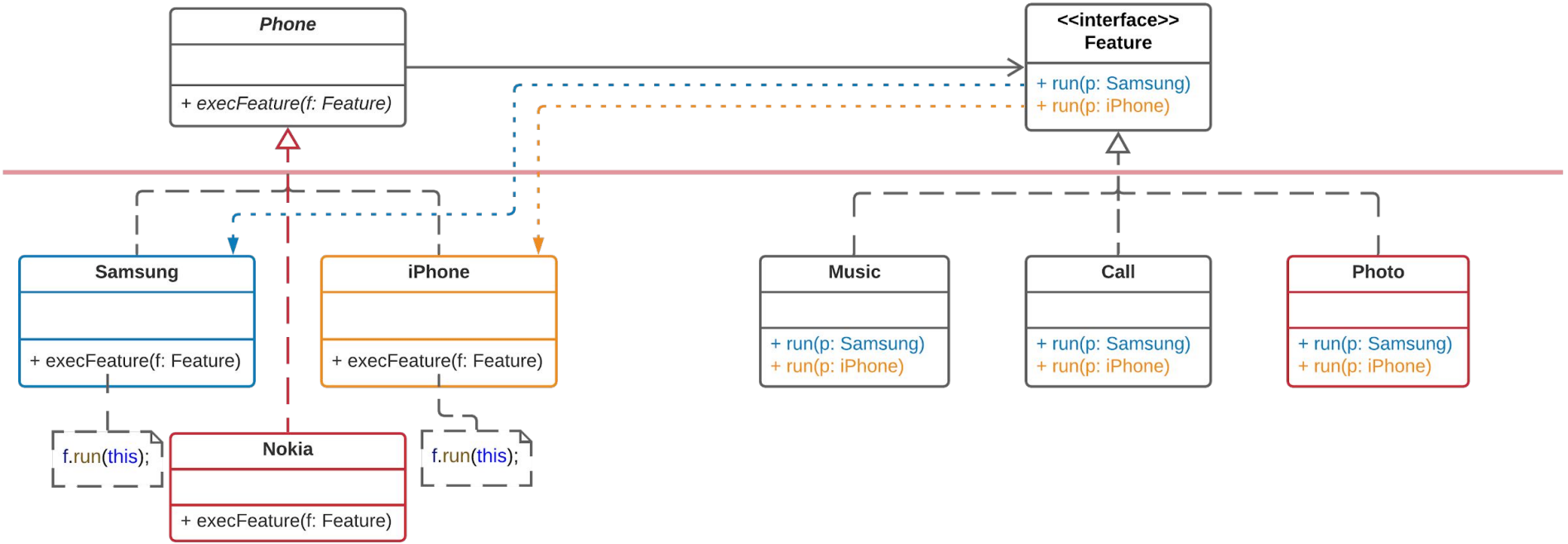
So Siri would violate LSP!





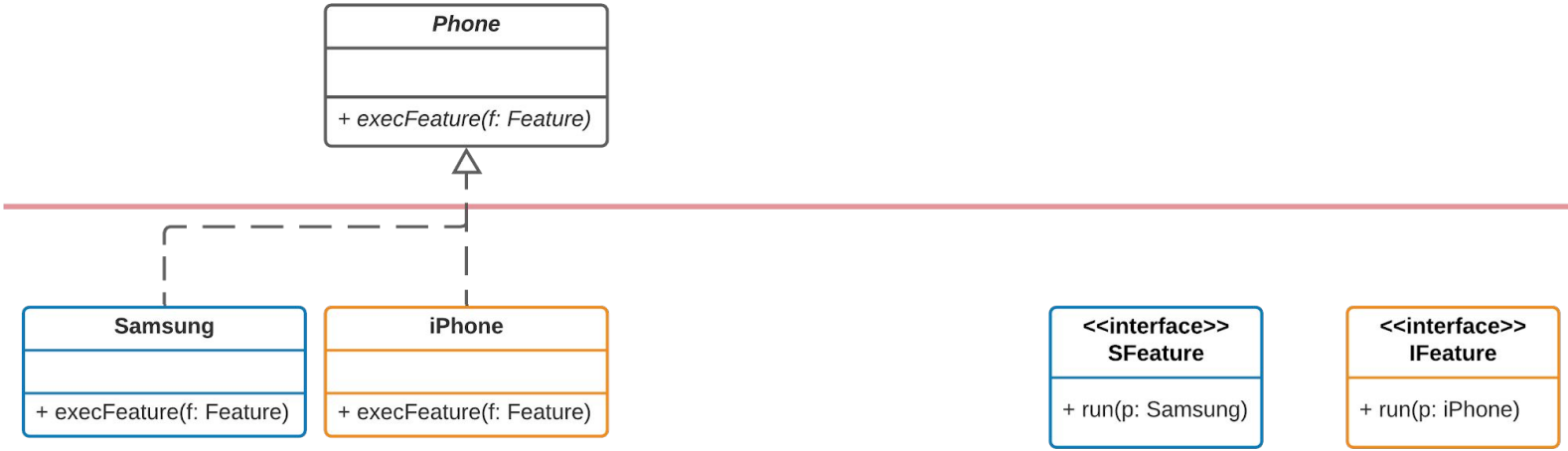
And also...how do we add Nokia???





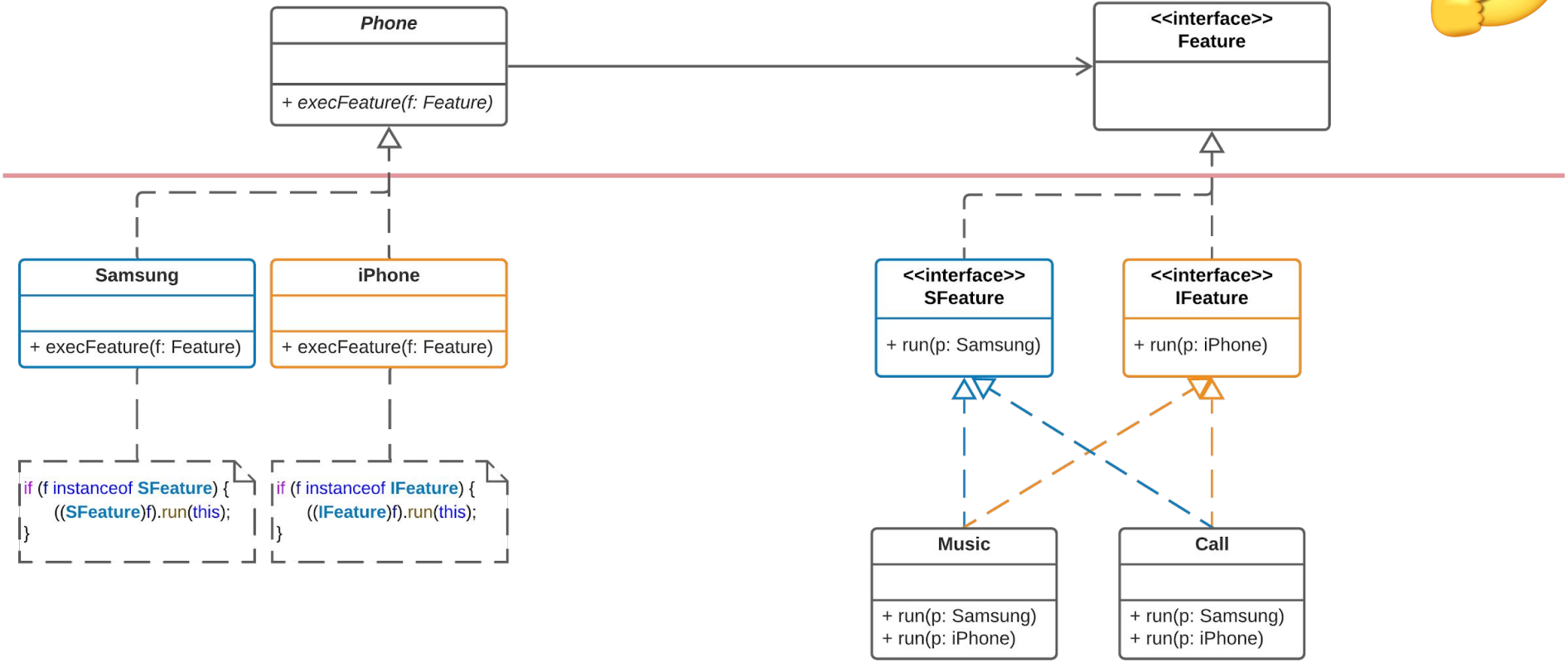
Segregate the interface!





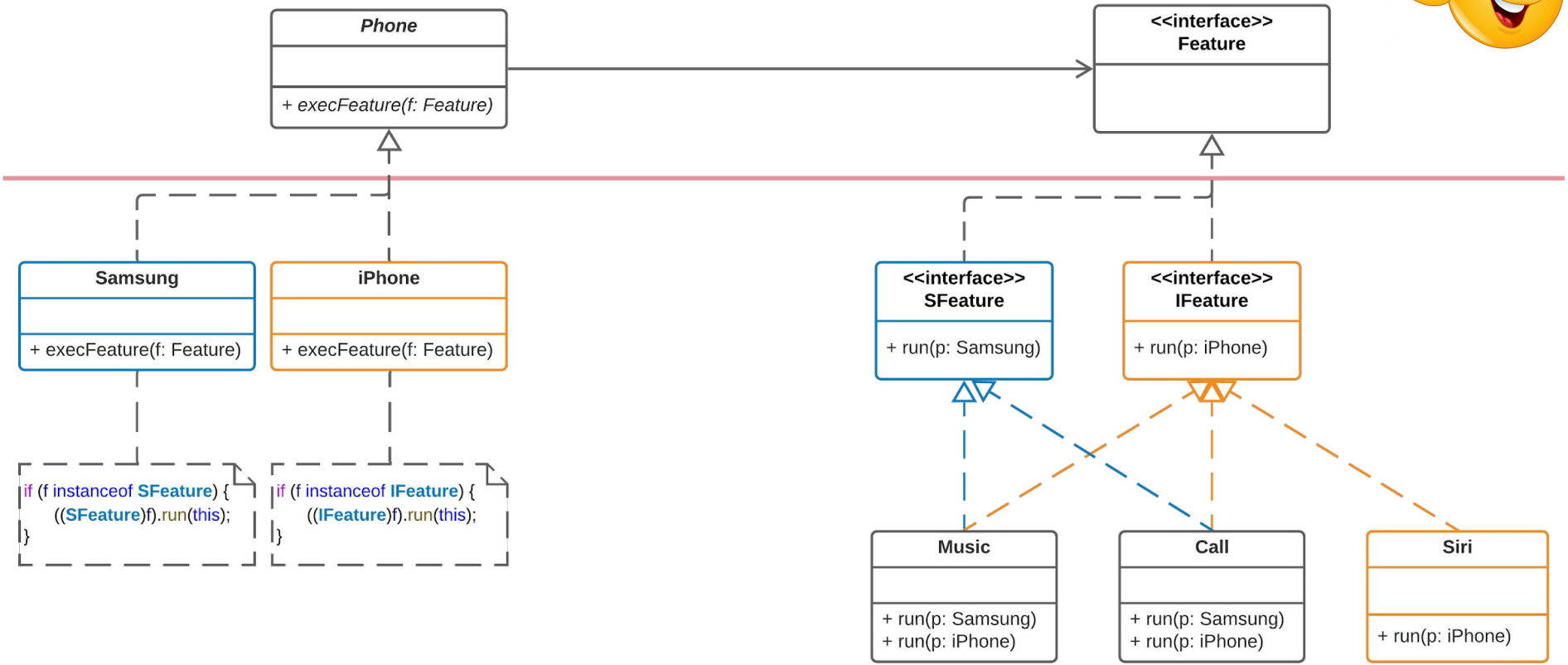
So now...how do we add Siri?





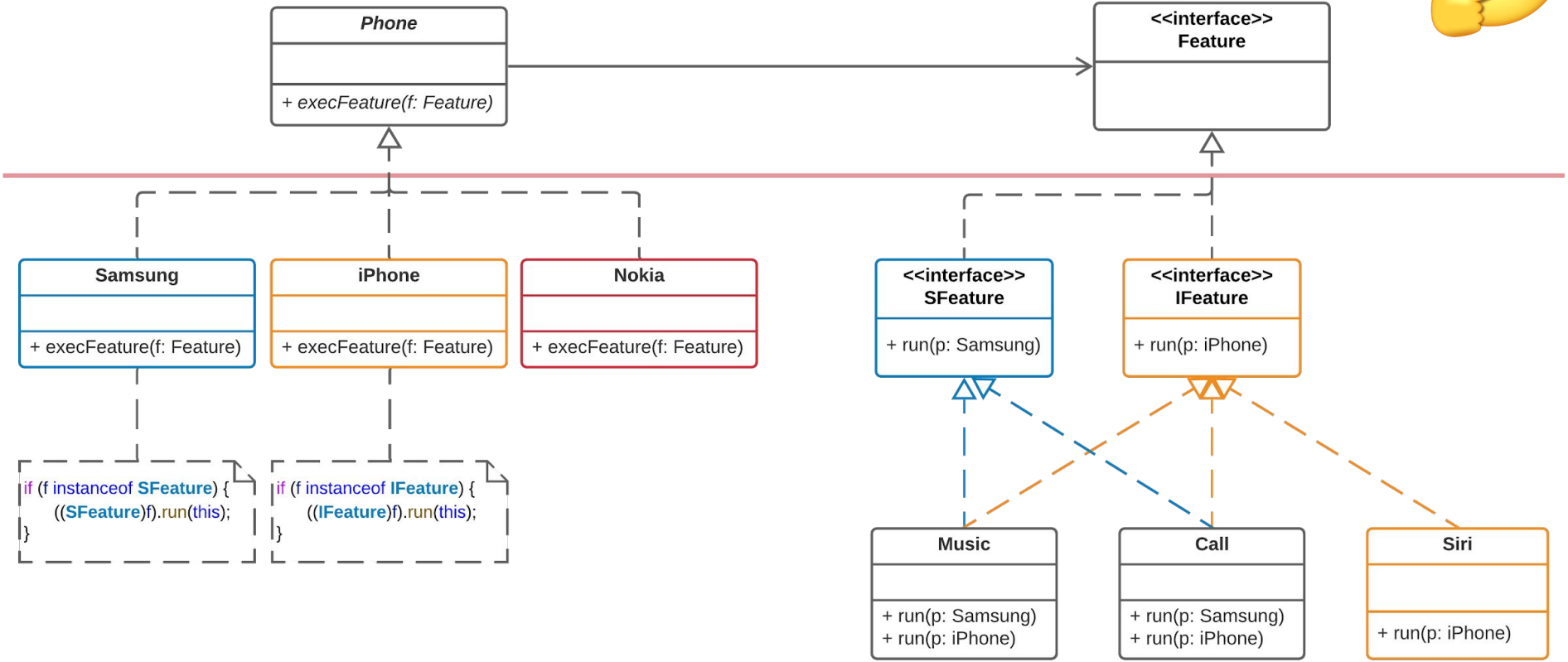
Simply implementing IFeature





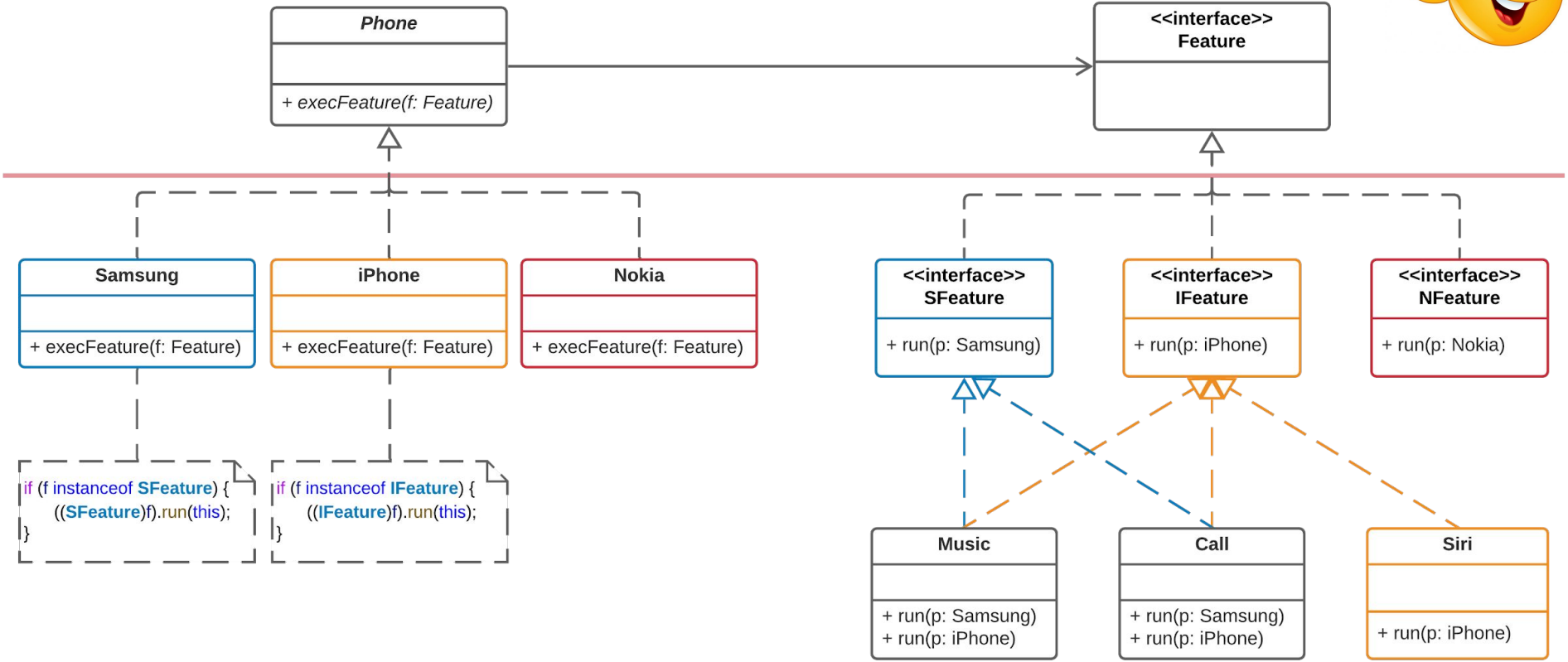
Ok...But how do we add Nokia???





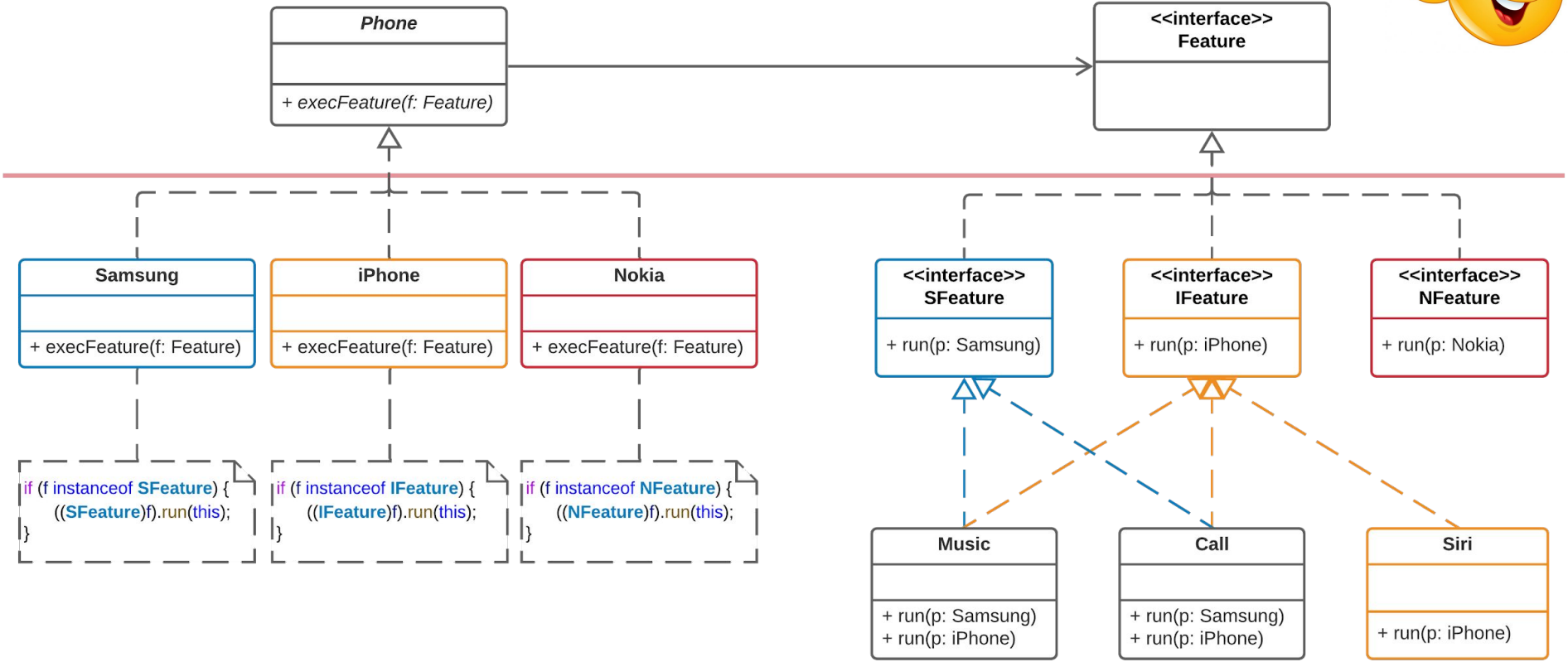
We add an interface...





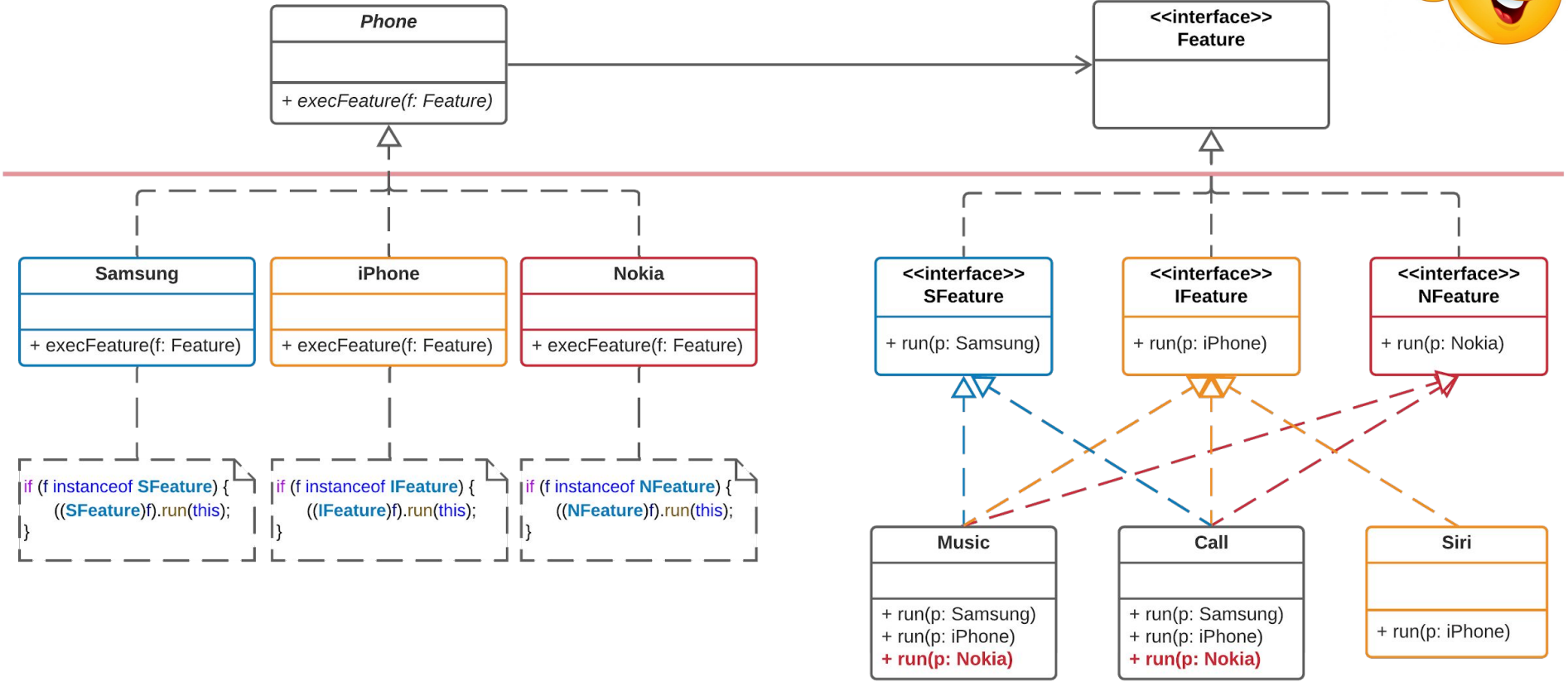
Add the code...




Phone
+ execFeature(f: Feature)
<<interface>>
Feature
Samsung
+ execFeature(f: Feature)
iPhone
+ execFeature(f: Feature)
Nokia
+ execFeature(f: Feature)
if (f instanceof SFeature) {
((SFeature)f).run(this);
}
if (f instanceof IFeature) {
((IFeature)f).run(this);
}
if (f instanceof NFeature) {
((NFeature)f).run(this);
}
<<interface>>
SFeature
+ run(p: Samsung)
<<interface>>
IFeature
+ run(p: iPhone)
<<interface>>
NFeature
+ run(p: Nokia)
Music
+ run(p: Samsung)
+ run(p: iPhone)
Call
+ run(p: Samsung)
+ run(p: iPhone)
Siri
+ run(p: iPhone)

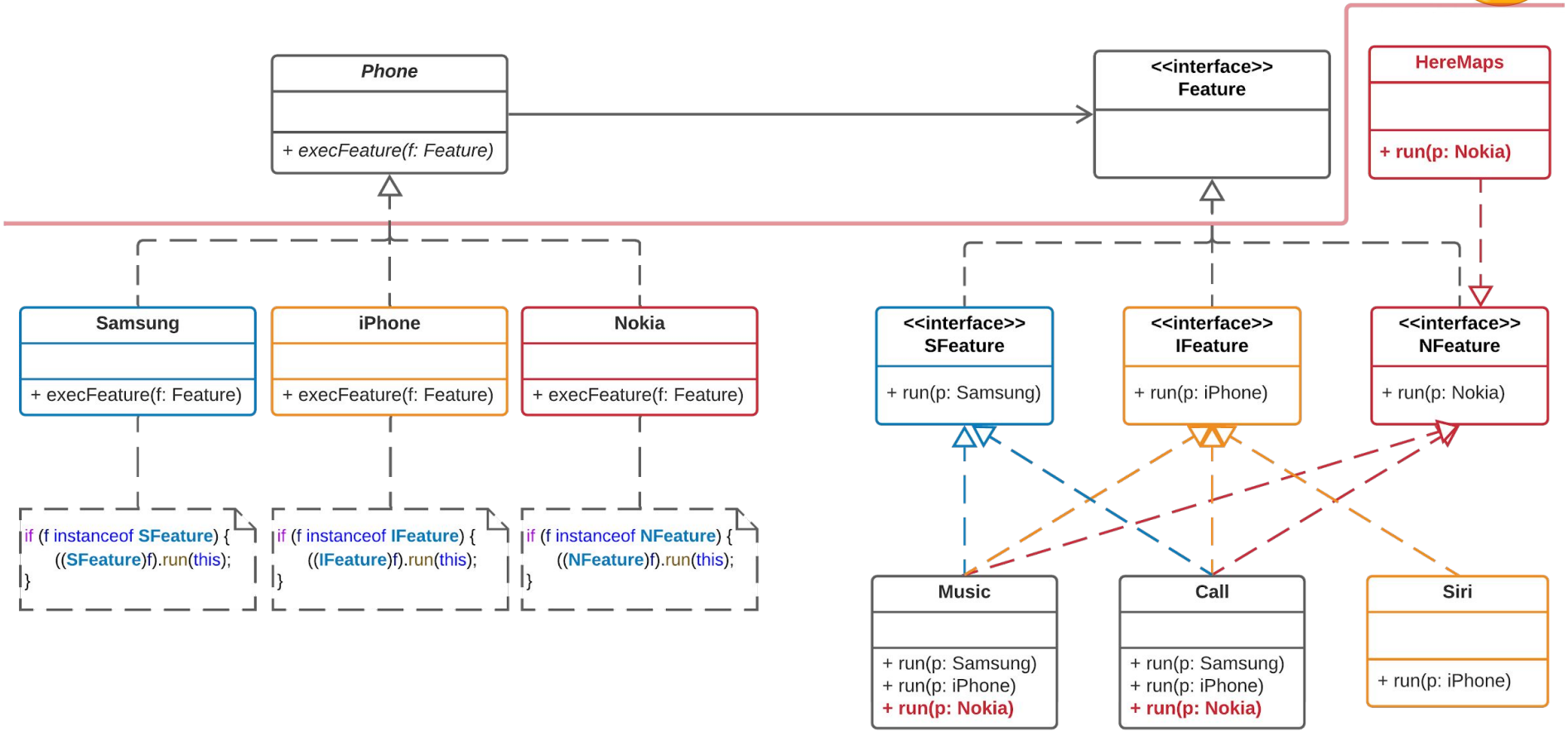
And any feature can choose to support Nokia





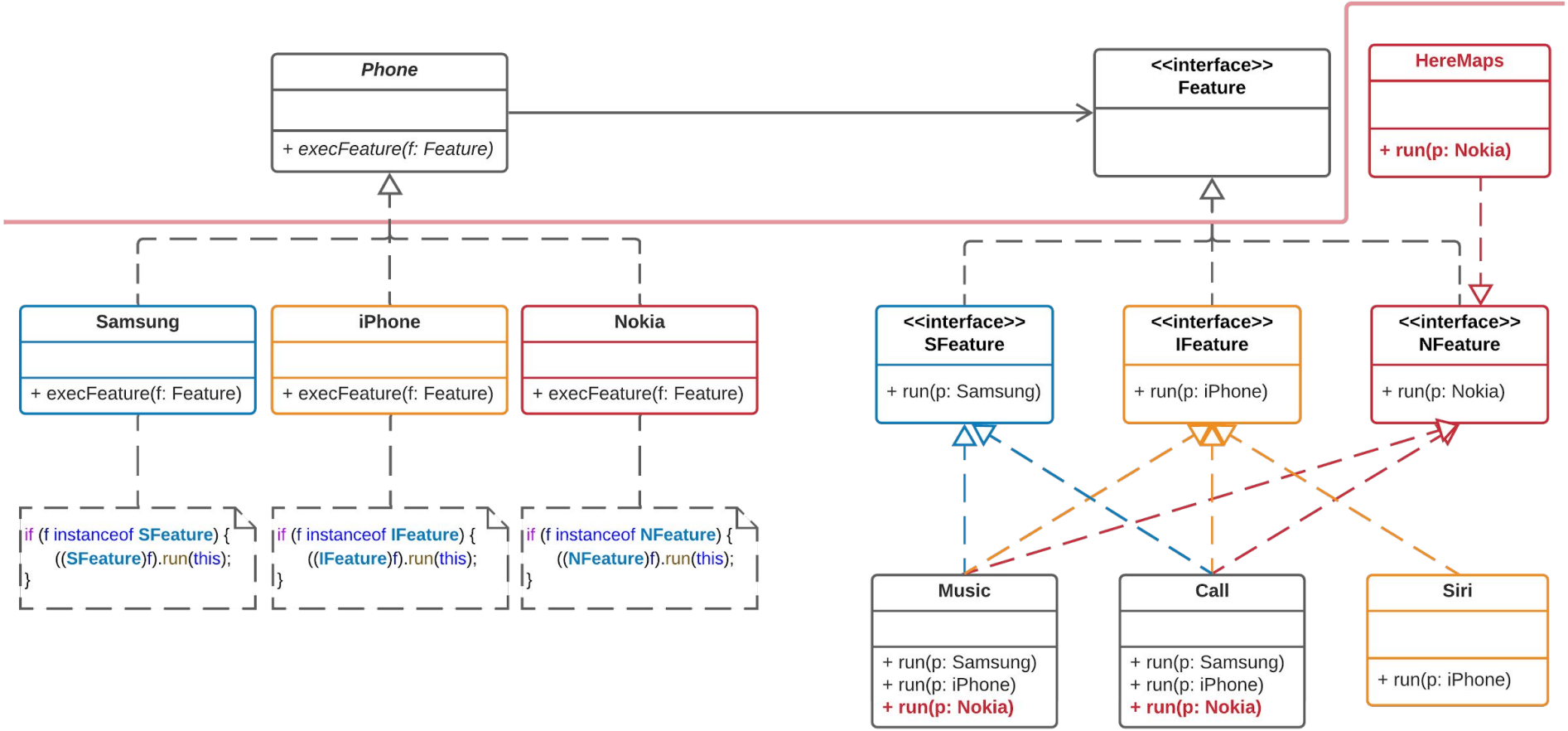
Or new features can support Nokia!





The Acyclic Visitor!





Behavioral Patterns Wrap up

Summary - Behavioral Patterns

- The Command Pattern encapsulates actions into objects
- The Visitor Pattern allows you to add any number of new “functions” / functionality to an existing class hierarchy, without modifying it.
- The Acyclic Visitor Pattern allows the same flexibility as the Visitor Pattern, while keeping the initial hierarchy open-closed to adding new subtypes.
- Template Method and Strategy Pattern solve the problem of having a small varying algorithm within a common behavior.
- Strategy Pattern allows for more flexibility (when it comes to reusing code), because it uses composition.
- Observer allows for objects to be notified when a specific async change event occurs.

Creational

- **Factory Method**
- **Abstract Factory**
- **Singleton**
- **Prototype (cloning)**
- **Builder**



Structural

- **Composite**
- **Decorator**
- Adapter
- Bridge
- Facade
- Proxy
- Flyweight



Behavioral

- **Chain of responsibility**
- **Command**
- **Observer**
- **Template Method**
- **Strategy Pattern**
- **Visitor (+ Acyclic Visitor)**
- Interpreter
- Iterator
- Mediator
- Memento
- State



Resources

- https://sourcemaking.com/design_patterns
- https://refactoring.guru/design-patterns
- ***Design Patterns: Elements of Reusable Object-Oriented Software***, written by *Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides*